

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторная работа №5 по курсу «Дискретный анализ»

Студент: М. А. Хорохорин
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 11.05.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №5

Задача: Необходимо найти самую длинную общую подстроку двух строк с использованием суффиксного дерева.

Вариант задачи: поиск наибольшей общей подстроки.

На вход подаются две строки. На первой строке вывода нужно распечатать длину максимальной общей подстроки, затем перечислить все возможные варианты общих подстрок этой длины в порядке лексикографического возрастания без повторов.

1 Описание

Требуется найти все наибольшие общие подстроки двух строк. Для этого строится обобщённое суффиксное дерево для строки $S_1\#S_2\$$, где вместо обычных символов-разделителей в реализации используются специальные коды 256 и 257. Поэтому служебные разделители не могут совпасть с символами входных строк.

Суффиксное дерево строится онлайн-алгоритмом Укконена за линейное время. После построения выполняется обход дерева в глубину. Для каждой вершины определяется, есть ли в её поддереве суффиксы первой строки и суффиксы второй строки. Если оба типа суффиксов присутствуют, то строка, соответствующая пути от корня до этой вершины, является общей подстрокой. Самая глубокая такая вершина задаёт длину наибольшей общей подстроки.

2 Исходный код

1 Структура дерева

Каждая вершина хранит переходы по символам, суффиксную ссылку и границы ребра в общей строке. Для листьев правый конец ребра равен специальному значению `LEAF_END`; фактический конец листового ребра задаётся текущей глобальной переменной `leafEnd`.

Листинг 1: Вершина суффиксного дерева

```
1 struct TNode {
2     std::map<int, int> children;
3     int suffixLink;
4     int start;
5     int end;
6
7     TNode(int start, int end);
8 };
```

Алфавит дерева целочисленный. Обычные байты входных строк попадают в диапазон $[0, 255]$, а разделители имеют значения 256 и 257.

Листинг 2: Построение общей строки

```
1 std::vector<int> BuildCombinedText(
2     const std::string &first,
3     const std::string &second
4 ) {
5     std::vector<int> result;
6     for (unsigned char ch : first) {
7         result.push_back(static_cast<int>(ch));
8     }
9     result.push_back(FIRST_SEPARATOR);
10    for (unsigned char ch : second) {
11        result.push_back(static_cast<int>(ch));
12    }
13    result.push_back(SECOND_SEPARATOR);
14    return result;
15 }
```

2 Алгоритм Укконена

При добавлении очередного символа поддерживаются активная вершина `activeNode`, активное ребро `activeEdge`, активная длина `activeLength` и число оставшихся суффиксов `remaining`. Если нужного перехода нет, создаётся лист. Если переход есть, но

следующий символ на ребре отличается, ребро расщепляется и создаётся внутренняя вершина. Суффиксные ссылки позволяют не начинать обработку каждого суффикса заново.

Листинг 3: Основной фрагмент Extend

```
1 | if (next == NO_CHILD) {
2 |     SetChild(activeNode, edgeCh, NewNode(pos, LEAF_END));
3 | } else {
4 |     int edgeLength = EdgeLen(next);
5 |     if (activeLength >= edgeLength) {
6 |         activeEdge += edgeLength;
7 |         activeLength -= edgeLength;
8 |         activeNode = next;
9 |         continue;
10 |    }
11 |
12 |    int splitCh = text[nodes[next].start + activeLength];
13 |    int curCh = text[pos];
14 |    if (splitCh == curCh) {
15 |        ++activeLength;
16 |        break;
17 |    }
18 |
19 |    int split = NewNode(
20 |        nodes[next].start,
21 |        nodes[next].start + activeLength - 1
22 |    );
23 |    SetChild(activeNode, edgeCh, split);
24 |    SetChild(split, curCh, NewNode(pos, LEAF_END));
25 |    nodes[next].start += activeLength;
26 |    SetChild(split, splitCh, next);
27 | }
```

3 Поиск ответа

Функция `Dfs` возвращает маску покрытия поддерева: есть ли в нём суффиксы первой строки, второй строки или обеих строк. Для восстановления самой подстроки дополнительно передаётся позиция одного суффикса из первой строки. Если вершина покрыта обеими строками, её глубина сравнивается с текущим максимумом, а соответствующая строка добавляется в `std::set`. Множество автоматически удаляет повторы и сортирует ответы лексикографически.

Листинг 4: Обработка общей подстроки в DFS

```
1 || if (coverage == COV_BOTH && idx != ROOT && depth > 0
```

```

2  ||    && firstStart != NO_CHILD && firstStart + depth <= sep) {
3  ||    if (depth > bestLen) {
4  ||        bestLen = depth;
5  ||        results.clear();
6  ||    }
7  ||    if (depth == bestLen) {
8  ||        results.insert(MakeString(firstStart, depth));
9  ||    }
10 || }

```

3 Консоль

```
# cmake -B build -DCMAKE_BUILD_TYPE=Release
# cmake --build build
# ./build/lab5/lab5_main
xabay
xabcbay
3
bay
xab
```

4 Тест производительности

1 Методика

Тест сравнивает построение обобщённого суффиксного дерева и поиск ответа с классическим динамическим программированием для задачи наибольшей общей подстроки. DP хранит таблицу размера $(|S_1|+1)(|S_2|+1)$ и работает за $O(|S_1| \cdot |S_2|)$ по времени и памяти. Суффиксное дерево строится за $O(|S_1| + |S_2|)$, после чего выполняется один DFS.

На малых размерах бенчмарк сравнивает полные множества ответов двух алгоритмов. На больших размерах DP пропускается, поскольку таблица становится слишком большой.

2 Результаты

$ S_1 = S_2 $	Суффиксное дерево, мкс	DP, мкс	Проверка
1 000	996	4 847	yes
2 000	1 329	24 298	yes
4 000	3 482	106 665	yes

Таблица 1: Сравнение с динамическим программированием, алфавит из 4 букв

$ S_1 = S_2 $	Суффиксное дерево, мкс	Длина ответа
10 000	12 783	12
50 000	208 406	15
100 000	339 247	17

Таблица 2: Большие тесты, DP пропущен

```
# python3 lab5/benchmark/generator.py 1000 1000 4 42
# ./build/lab5/lab5_benchmark <tests.txt
Suffix tree LCS time: 996us
LCS length: 10
LCS count: 1
DP LCS time: 4847us
DP LCS length: 10
DP LCS count: 1
Results equal: yes
```

```
# python3 lab5/benchmark/generator.py 4000 4000 4 42
# ./build/lab5/lab5_benchmark <tests.txt
Suffix tree LCS time: 3482us
LCS length: 12
LCS count: 1
DP LCS time: 106665us
DP LCS length: 12
DP LCS count: 1
Results equal: yes
```

```
# python3 lab5/benchmark/generator.py 100000 100000 4 42
# ./build/lab5/lab5_benchmark <tests.txt
Suffix tree LCS time: 339247us
LCS length: 17
LCS count: 1
DP LCS skipped: input is too large
```

3 Анализ результатов

На малых тестах суффиксное дерево уже быстрее DP: при длине строк 4 000 выигрыш составляет примерно **30 раз**. При дальнейшем росте размера DP становится непрактичным из-за квадратичной таблицы, тогда как суффиксное дерево продолжает работать в пределах ограничений. Бенчмарк также проверяет корректность: для малых тестов множества найденных подстрок у дерева и DP совпадают.

5 Дневник выполнения работы

- 11.05.2026: изучено задание на поиск наибольшей общей подстроки двух строк с использованием суффиксного дерева.
- 11.05.2026: реализован алгоритм Укконена для обобщённой строки с двумя уникальными разделителями, отсутствующими во входном алфавите.
- 11.05.2026: добавлен обход дерева с масками принадлежности суффиксов первой и второй строкам, сбор ответов выполняется через `std::set`.
- 11.05.2026: написаны генератор и benchmark, результат суффиксного дерева сверяется с динамическим программированием на допустимых размерах.

6 Выводы

Выполнив данную лабораторную работу по курсу «Дискретный анализ», я реализовал поиск наибольшей общей подстроки двух строк с помощью обобщённого суффиксного дерева. Основная часть работы заключалась в реализации онлайн-алгоритма Укконена: активной точки, остатка необработанных суффиксов, расщепления рёбер и суффиксных ссылок.

Для поиска ответа дерево обходится в глубину, а каждая вершина помечается по наличию суффиксов первой и второй строки в её поддереве. Самая глубокая вершина с покрытием обеих строк задаёт длину ответа, а множество `std::set` обеспечивает вывод без повторов и в лексикографическом порядке.

Тестирование подтвердило преимущество суффиксного дерева над динамическим программированием: на строках длины 4000 дерево работает примерно в **30 раз** быстрее, а на больших размерах DP уже не используется из-за квадратичной памяти.

Список литературы

- [1] Ukkonen, E. *On-line construction of suffix trees*. Algorithmica, 14(3):249–260, 1995.
- [2] Gusfield, D. *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.
- [3] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание*. — Издательский дом «Вильямс», 2007.